

Hochschule
Hof
University of
Applied Sciences

MoProCo-Agentic

DSL-Guided Agentic Coding for Full-Stack Web Applications

Thomas Buchmann

Hochschule Hof

MODELSWARD 2027

Agenda

1. Motivation and problem statement
2. The MoProCo-Agentic approach
3. Case studies
4. Evaluation
5. Discussion and conclusion

Introduction

Agentic coding today

- LLM coding agents build complete applications from prose descriptions
- Steering happens via natural language guardrail documents (`AGENTS.md`, Cursor Rules)
- The agent plans, writes, and *tests* its own code

The appeal

Minimal upfront modeling, fast prototyping, full-stack output from a single prompt

The problem with natural language guardrails

- Verbose and unvalidatable: nothing checks that the document is complete, consistent, or still true
- No control over test authorship: the agent writes the tests for the features it implements and may satisfy its own tests while ignoring requirements
- Token cost: every entity, DTO, controller, and migration is generated token by token, and the agent repeatedly reloads the whole codebase as context

Core idea: a strict division of labor

- **Deterministic M2T generation** for everything structurally predictable: entities, repositories, DTOs, migrations, REST controllers, React Query hooks, TypeScript interfaces, Zod schemas, test skeletons
- **Agentic coding** for what needs domain judgment: business logic in service classes, UI event handlers, test assertions
- The DSLs are the *living specification*: the source of truth for all generated artifacts, not a document that can drift

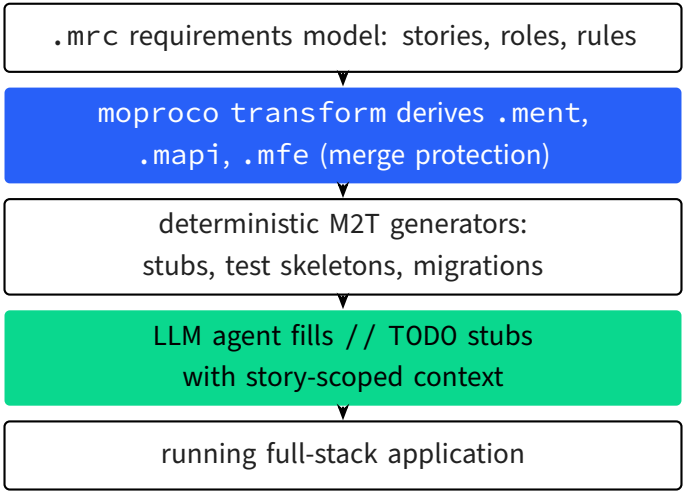
The MoProCo-Agentic Approach

A family of four interrelated DSLs

DSL	Content
`.mrc`	Requirements: stories, roles, acceptance criteria, business rules
`.ment`	Entities: JPA persistence model, validations, derived attributes
`.mapi`	REST API: endpoints on entities, pagination, filtering
`.mfe`	Frontend: pages bound to endpoints, dot notation for nested fields

- Implemented in Langium, full IDE support
- Cross-references are formally validated: an endpoint referencing a non-existent entity is a grammar error at authoring time

The transformation and generation pipeline



What gets generated

Backend (Spring Boot, Java 21)

- JPA entities, repositories, DTOs
- REST controllers, Flyway migrations
- JUnit persistence and validation test skeletons

Frontend (React)

- TypeScript interfaces, Zod schemas
- React Query hooks, pages, API client
- Playwright E2E stubs

Key property

One `// TODO` assertion per acceptance criterion declared in the `.mrc` file: the test structure is a direct projection of the requirements model

The structured agent interface

- An opencode plugin exposes two DSL-backed tools:
  - `moproco_list_stories`: enumerate the stories
  - `moproco_story_context`: story-scoped context bundle
- The context bundle contains the originating story, its acceptance criteria, the involved entity definitions, the service stub, and the matching test skeleton, in one coherent context window
- The agent works *inside* the typed scaffold instead of inventing structure

Case Studies and Evaluation

Two case studies, two purposes

Blog application: step-by-step walkthrough

- Posts and comments, two business rules (non-empty title, comment length limit of 2000 characters)
- Demonstrates the basic workflow end-to-end

Order management system: evaluation vehicle

- B2B domain: enumerations, roles, derived attributes, pagination, filtering, dot notation
- Subject of the quantitative evaluation

The requirements DSL in practice

```
enum OrderStatus { open, waitingForStock, packaged, shipped, canceled, returned }

role SalesClerk {
  create:Order, read:Order, update:Order
  create:OrderLine, update:OrderLine, delete:OrderLine
}

story CreateOrder {
  as [SalesClerk]
  iWant "to create an order with all required attributes"
  soThat "stock is reduced"
  acceptance {
    given "I am logged in as a SalesClerk"
    when "I submit an order with line items"
    then "the order is created and stock is updated"
  }
}
```

From stub to implementation

```java
// Generated stub (excerpt): signature, types, TODO in place
public PostDto createPost(PostDto dto) { /* TODO */ }

// Agent-completed implementation (excerpt)
public PostDto createPost(PostDto dto) {
    PostEntity entity = postMapper.toEntity(dto);
    entity.setId(null);   // let the database assign the ID
    postRepository.save(entity);
    return postMapper.toDto(entity);
}
```

- The agent received the stub, the entity definition, the story with its acceptance criteria, and the Playwright stub in one context
- No inference of entity structure, API contracts, or test setup was necessary

Evaluation setup

- Task: build the order management application from scratch
- **MoProCo-Agentic** vs. **LLM-only baseline**: same model (GLM-5.2), same tech stack, no DSL input, no generated scaffolding
- 5 runs per setting, median values reported
- Deterministic code generation produced bit-identical output in all runs
- All tests passed in both settings (100% pass rate), but coverage differs substantially

Results: code composition

	Files	LOC	Agent LOC
Backend	60	1,811	7
Frontend	26	597	53
Total	**86**	**2,403**	**60**

97.7% of the code was produced deterministically, the agent contributed 2.5% (7 backend + 53 frontend LOC); all // TODO stubs were filled, zero stub LOC remained

Results: token economy

Metric	MoProCo-Agentic	LLM-only
Input tokens	2,785,390	10,935,593
Output tokens	20,190	71,076
Total tokens	2,805,580	16,442,521
Provider steps	57	292
Subagents	0	4
Tests	33 (BE+FE)	16 (BE)

- 5.9× fewer tokens, 5.1× fewer steps, 3.9× less input
- The baseline needed 4 subagent sessions; MoProCo-Agentic needed none

Discussion and Conclusion

Strengths and positioning

Strengths

- Reduced degrees of freedom: pre-typed stubs make errors easier to detect and correct
- DSLs as living specification, no drift
- Developer control over test authorship

vs. schema-first tooling (OpenAPI Generator)

- Full vertical scope instead of API layer only
- Formal traceability from requirements to tests
- Structured agent interface
- Transformation pipeline for bootstrapping

Limitations and threats to validity

- Single tech stack (Spring Boot, React); other stacks need new generators
- Partial incremental generation: merge protection, but no DSL-level diff tracking yet
- Incomplete semantic validation across the DSL family
- One-way transformation: no round-trip synchronization yet
- Plain HTML frontend, no UI component library

Threats to validity

Both case studies are well-structured domains; scalability to exploratory domains and uncertain entity models is unproven; a controlled user study is needed for stronger empirical claims

Conclusion

- MoProCo-Agentic combines deterministic M2T transformation with agentic coding for full-stack web applications
- Four DSLs replace natural language guardrails with a formal, validatable specification; test structure becomes a projection of the requirements model
- In the evaluation: 97.7% of the code generated deterministically, 5.9× fewer tokens, 5.1× fewer steps, and more tests than the LLM-only baseline
- Future work: pluggable generator backends, full incremental generation, bidirectional synchronization, a user study, deeper semantic validation, UI component library integration

Thank you!

Resources

- Toolchain incl. DSL files of both case studies:
  `https://github.com/tbuchmann/moproco`
- Generated order management case study:
  `https://github.com/tbuchmann/order-management`

Questions?